Cleaning Platform

Microservice Web Application

J2EE Enterprise Application Development
Humber College — Semester 4

Team Member	Role
Maksim Ovchinnikov	Team Lead / Backend Engineer
Harpreet Kaur	Frontend Developer
Pragati Sahani	QA / Tester

April 2026

Table of Contents

1. Project Overview
2. Team Contributions
3. System Architecture
4. Service Descriptions
   4.1 IAC Service (Identity & Access Control)
   4.2 Property Service
   4.3 Pricing Service
   4.4 Order Service
   4.5 Booking Service
   4.6 UI Service
5. Domain Model
6. Messaging Architecture (Kafka + Outbox Pattern)
7. Authentication & Security
8. API Reference
9. Sequence Diagrams
10. Postman Collection
11. Deployment
12. Testing
13. Conclusion

1. Project Overview

The **Cleaning Platform** is a full-stack, microservice-based web application built with Java, Spring Boot 3, and Docker Compose. It models a real-world cleaning services marketplace where **clients** can register, add properties, and book cleaning orders; **cleaners** can view their schedule and log completions; and **administrators** can manage users and roles.

The application demonstrates the following enterprise Java concepts:

- Microservice decomposition with clear bounded contexts per service
- JWT-based stateless authentication with refresh token rotation
- Inter-service REST communication (Order→Pricing, Order→Property)
- Spring Security with method-level RBAC (@PreAuthorize)
- Spring Data JPA with PostgreSQL and Flyway DB migrations
- Docker Compose orchestration with health checks and shared networking
- Thymeleaf-based UI gateway acting as a reverse proxy
- RESTful API design with OpenAPI (Swagger) documentation

Technology Stack

Layer	Technology
Language	Java 17+
Framework	Spring Boot 3.x
Security	Spring Security + JWT (JJWT / custom)
Database	PostgreSQL 16
ORM	Spring Data JPA + Hibernate
Migrations	Flyway
Build	Apache Maven
Containerization	Docker + Docker Compose
Frontend	Thymeleaf + Bootstrap CSS
API Docs	SpringDoc OpenAPI (Swagger UI)
Messaging	Apache Kafka 3.x (Confluent 7.6) + Transactional Outbox
Functional style	Vavr (Either, Option)

2. Team Contributions

The project was developed collaboratively by a three-person team, with each member owning a distinct area of the system:

Maksim Ovchinnikov — Team Lead / Backend Engineer

- Designed and implemented the overall microservice architecture
- Developed IAC Service: JWT authentication, role management, token rotation, super admin seeding
- Developed Property Service: property CRUD, ownership validation, internal validate endpoint
- Developed Pricing Service: service type and addon catalog, PricingEngine with frequency discounts
- Developed Order Service: full order lifecycle, round-robin cleaner assignment, earnings tracking
- Developed Booking Service: integrated facade for properties, orders, catalog and earnings
- Configured Docker Compose: networking, health checks, environment variable injection
- Created Flyway database migrations for all services
- Set up Postman collection with automated test scripts and token management
- Authored README and project documentation

Harpreet Kaur — Frontend Developer

- Designed and implemented the UI Service (Thymeleaf + Spring MVC)
- Built the web gateway with reverse proxy routing to backend services
- Created login, registration, dashboard, properties, orders, and catalog pages
- Implemented client-side form validation and error handling
- Styled the application with Bootstrap and custom CSS
- Integrated frontend with IAC authentication flow (session/token management)
- Implemented role-based menu/page visibility in the UI

Pragati Sahani — QA / Tester

- Designed and executed the test plan covering all microservices
- Performed functional testing of all REST API endpoints using Postman
- Validated authentication flows: registration, login, token refresh, logout

- Tested role-based access control (RBAC) restrictions and edge cases
- Executed negative testing: invalid inputs, expired tokens, unauthorized access
- Validated order lifecycle: create → pay → complete → cancel
- Tested cleaner assignment round-robin behavior with multiple cleaners
- Verified pricing calculations for all service types, addons, and frequency discounts
- Documented test results and reported bugs to the development team

3. System Architecture

The platform follows a **microservice architecture** where each service owns its domain, has its own Spring Boot application, and communicates with others via HTTP REST calls (for synchronous, request-response flows) and **Apache Kafka** (for asynchronous, event-driven flows). All services share a single PostgreSQL instance (with separate schemas) and communicate over a dedicated Docker bridge network (cleaning-network).

The **Transactional Outbox Pattern** ensures reliable event delivery: events are written to an outbox_events table in the same database transaction as the business operation. A scheduled relay process polls the table and publishes events to Kafka, guaranteeing at-least-once delivery even if the service crashes between the write and the Kafka send.

Service Port Map

Service	Host Port	Container Port	Role
PostgreSQL	15432	5432	Shared relational database
Zookeeper	12181	2181	Kafka coordination
Kafka Broker	19092	9092	Async event bus
IAC Service	18080	8080	Identity & Access Control
Property Service	18081	8080	Client property management
Pricing Service	18082	8080	Catalog & price calculation
Order Service	18083	8080	Order lifecycle & cleaners
UI / Booking Service	18090	8090	Frontend + reverse proxy

Component Diagram

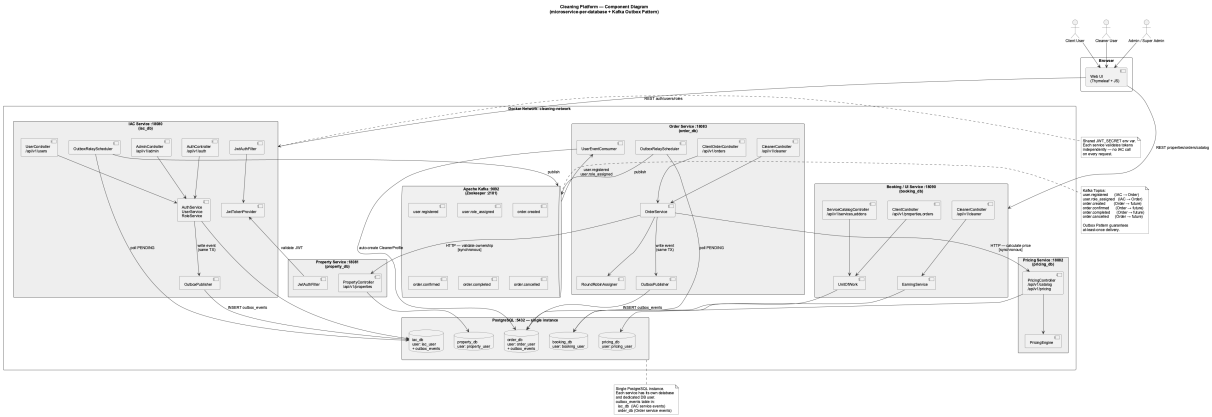

Figure: Full system component diagram showing inter-service communication

4. Service Descriptions

4.1 IAC Service — Identity & Access Control

The IAC Service is the security backbone of the platform. It manages user registration, authentication, JWT token issuance, refresh token rotation, and role-based access control. A super admin account is automatically seeded at startup via the SuperAdminSeeder component.

Key Features:

- User registration with BCrypt password hashing
- JWT access tokens (15 min) + refresh tokens (7 days) with single-use rotation
- Roles: USER, CLIENT, CLEANER, ADMIN, SUPER_ADMIN
- All authenticated requests validated by JwtAuthFilter
- Admin endpoints: assign/revoke roles, list/delete users

Entities:

UserEntity: id, username, email, passwordHash, firstName, lastName, enabled, createdAt, updatedAt, roles (M:M), refreshTokens (1:M)

RoleEntity: id, name

RefreshTokenEntity: id, token, user, expiresAt

4.2 Property Service

Manages physical properties owned by clients. Each property represents a location to be cleaned (apartment, house, or office). Properties are identified by client ID (UUID derived from username) and support soft-deletion (active flag).

Key Features:

- Property types: APARTMENT, HOUSE, OFFICE
- Area validation: minimum 10 sqm, bathrooms: 1–10
- Internal validation endpoint used by Order Service to confirm ownership
- Soft delete — deactivated properties excluded from lists

4.3 Pricing Service

Provides a public service catalog (types and addons) and a price calculation engine. The PricingEngine computes the total price based on area, bathroom count, service type, selected addons, and frequency discount.

Pricing Formula:

```
basePrice = areaSqm × ratePerSqm + bathroomsCount × ratePerBathroom
addonsPrice = sum(selectedAddon.price)
discount = frequencyDiscount (WEEKLY=10%, BI_WEEKLY=5%, MONTHLY=15%)
totalPrice = (basePrice + addonsPrice) × (1 - discount)
```

Service Types:

Code	Name	Rate/sqm	Rate/bathroom
STANDARD	Standard Cleaning	$0.50	$15.00
DEEP	Deep Cleaning	$1.20	$25.00
MOVE_IN_OUT	Move In/Out Cleaning	$1.80	$35.00

Sample Addons:

Code	Name	Price
OVEN	Oven Cleaning	$40.00
FRIDGE	Fridge Cleaning	$35.00
WINDOWS	Window Cleaning	$50.00
LAUNDRY	Laundry & Folding	$30.00
IRONING	Ironing	$25.00

4.4 Order Service

The Order Service handles the complete lifecycle of cleaning orders, from creation through payment to completion. It communicates with Property Service (ownership validation) and Pricing Service (price calculation) via HTTP. Cleaner assignment uses a **round-robin algorithm** based on the cleaner's last assignment timestamp.

Order Status Flow:

```
PENDING → CONFIRMED → COMPLETED (or any status → CANCELLED)
```

Round-Robin Assignment:

When an order is paid, the system selects the available cleaner who was least recently assigned (ordered by lastAssignedAt ASC). If no cleaners are available, the field is left null and the admin must manually assign.

Earnings:

Upon order completion, an EarningEntity is created for the cleaner with 70% of the order's total price.

4.5 Booking Service

The Booking Service is an integrated facade that combines property management, order management, and the service catalog into a single Spring Boot application. It uses its own PostgreSQL schema (booking) and is the primary backend used by the UI Service.

Controllers:

- ServiceCatalogController — public: /api/v1/services, /api/v1/addons
- ClientController — ROLE_CLIENT: /api/v1/properties, /api/v1/orders
- CleanerController — ROLE_CLEANER: /api/v1/cleaner/*

4.6 UI Service

The UI Service is a Spring Boot application with Thymeleaf templating that serves the web frontend and acts as a reverse proxy to the backend services. It maintains user session state after authentication and provides role-based navigation.

- ProxyController: routes API calls from the browser to the appropriate backend service
- HomeController: serves Thymeleaf pages (dashboard, login, properties, orders, catalog)
- Session management: stores JWT token in HTTP session after login

5. Domain Model

The following class diagram shows the core domain entities across all services and their relationships. Note that while services share a PostgreSQL instance, each service manages its own schema and entities independently.

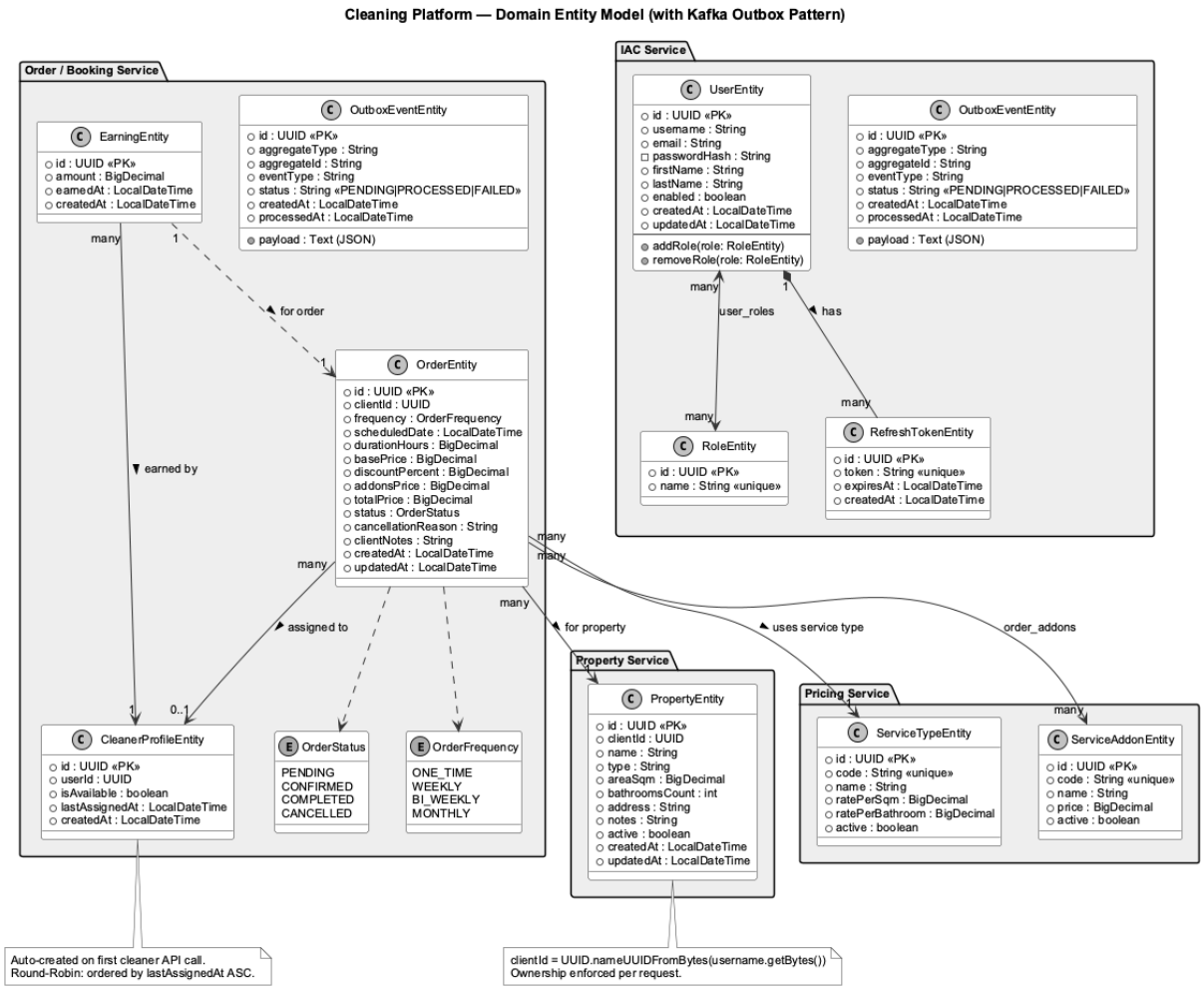

Figure: Domain Entity Model — all services

6. Messaging Architecture — Kafka + Transactional Outbox

To achieve **reliable, decoupled inter-service communication**, the platform uses **Apache Kafka** as the event bus combined with the **Transactional Outbox Pattern**. This guarantees that events are never lost — even if a service crashes between processing a request and publishing to Kafka.

Outbox Pattern — How It Works

- **Step 1 (Atomic):** Business logic writes both the domain record AND an outbox_events row in a single DB transaction
- **Step 2 (Async):** OutboxRelayScheduler polls outbox_events WHERE status=PENDING every 5 seconds
- **Step 3 (Publish):** Relay publishes each event to Kafka and marks it PROCESSED
- **Retry:** On Kafka failure, event stays PENDING and is retried on next poll cycle

Kafka Topics

Topic	Producer	Consumer(s)	Trigger
user.registered	IAC Service	Order Service	New user registration
user.role_assigned	IAC Service	Order Service	Admin assigns CLEANER role
user.role_revoked	IAC Service	(future)	Admin revokes a role
order.created	Order Service	(future notifications)	Order placed by client
order.confirmed	Order Service	(future notifications)	Order paid + cleaner assigned
order.completed	Order Service	(future analytics)	Cleaner marks order complete
order.cancelled	Order Service	(future notifications)	Client cancels order

Consumer: Order Service — UserEventConsumer

The Order Service consumes user.registered and user.role_assigned events to automatically create a CleanerProfile for any user who registers as or is assigned the CLEANER role. This eliminates the previous dependency on the cleaner making their first API call before being available for round-robin assignment.

Outbox Pattern Sequence

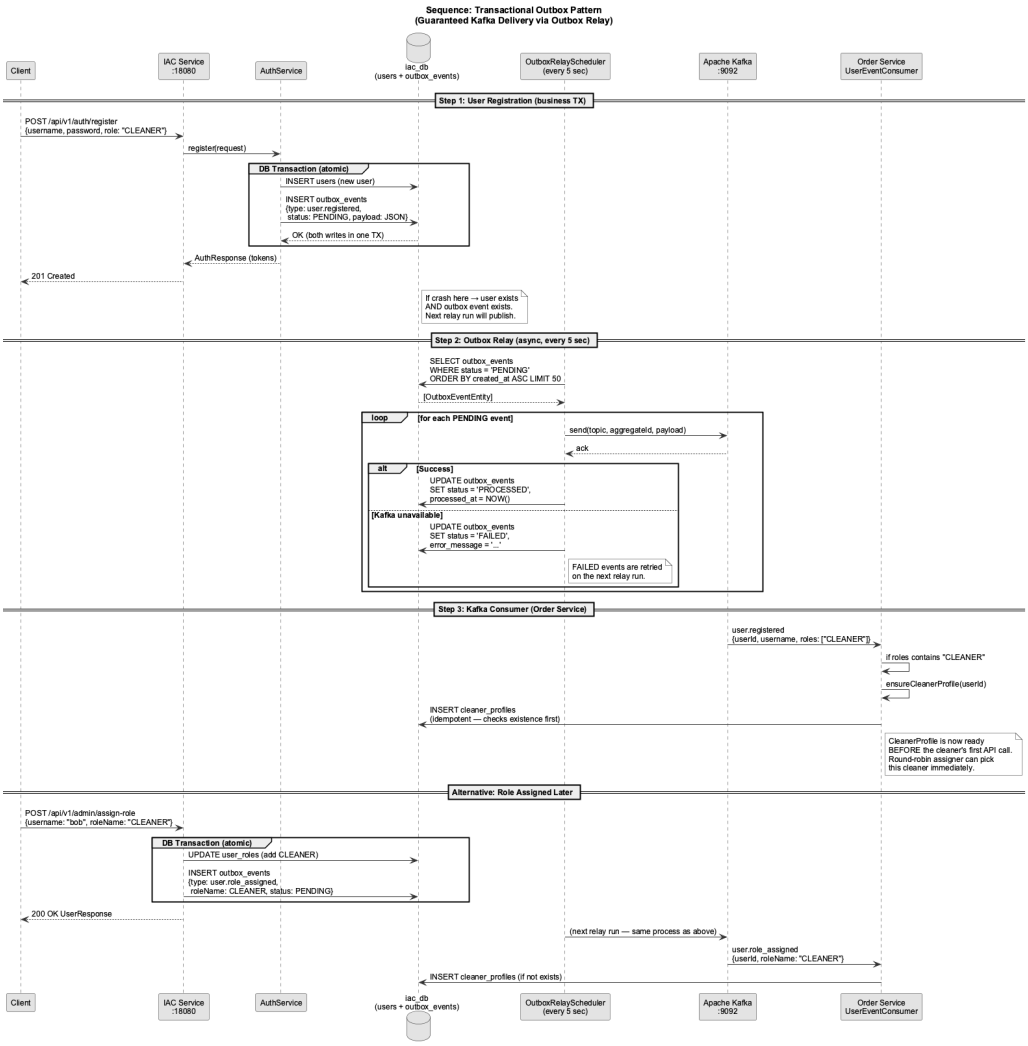

Figure: Transactional Outbox Pattern — from business transaction to Kafka delivery

7. Authentication & Security

All services (except public catalog endpoints) are protected by JWT Bearer token authentication. The same JWT secret is shared across all services via the JWT_SECRET environment variable, allowing any service to independently validate tokens issued by the IAC Service.

Authentication Flow

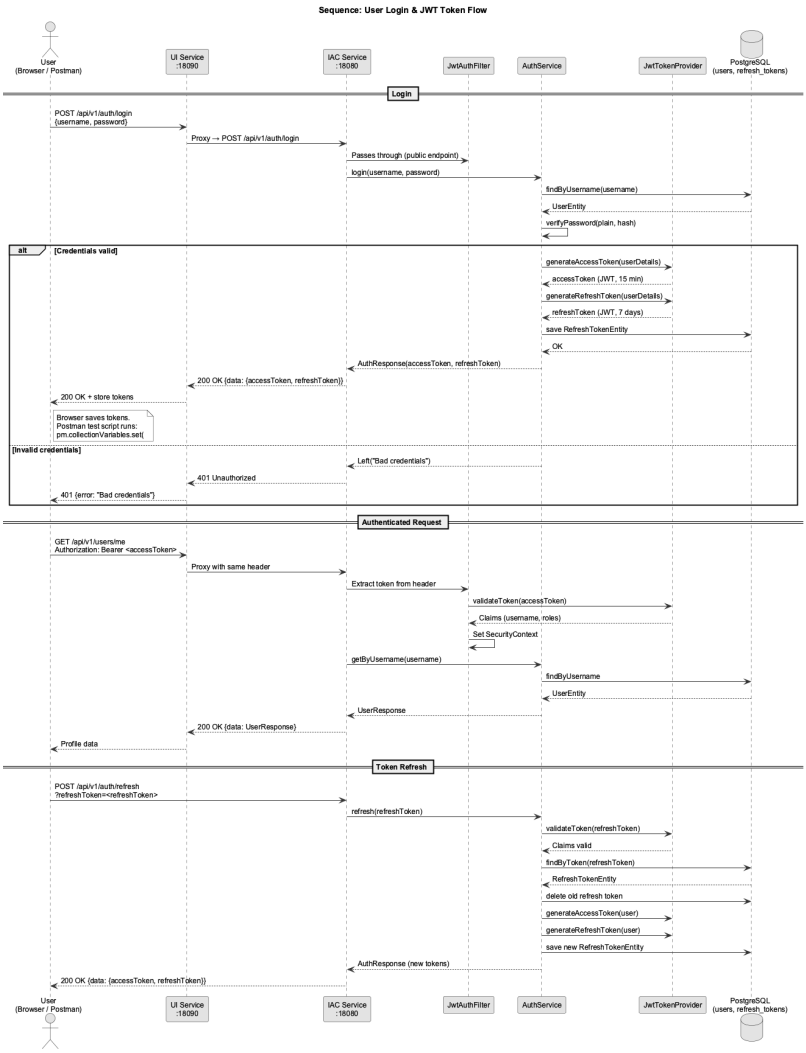

Figure: Login and JWT token flow — full sequence

Security Configuration

Public endpoints (no auth required):

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- GET /api/v1/catalog/services
- GET /api/v1/catalog/addons
- GET /api/v1/services
- GET /api/v1/addons
- GET /swagger-ui/** and /v3/api-docs/**

All other endpoints require a valid JWT Bearer token in the Authorization header: Authorization: Bearer <accessToken>. Method-level security is enforced with @PreAuthorize annotations.

7. API Reference

IAC Service (:18080)

Method	Path	Auth	Description
POST	/api/v1/auth/register	None	Register new user
POST	/api/v1/auth/login	None	Login, returns JWT tokens
POST	/api/v1/auth/refresh	None	Refresh access token
POST	/api/v1/auth/logout	Bearer	Revoke all refresh tokens
GET	/api/v1/users/me	Bearer	Get own profile
GET	/api/v1/users	ADMIN+	List users (paginated)
GET	/api/v1/users/{id}	ADMIN+	Get user by UUID
DELETE	/api/v1/users/{id}	SUPER_ADMIN	Delete user
GET	/api/v1/admin/roles	ADMIN+	List available roles
POST	/api/v1/admin/assign-role	ADMIN+	Assign role to user
POST	/api/v1/admin/revoke-role	ADMIN+	Revoke role from user

Property Service (:18081)

Method	Path	Auth	Description
POST	/api/v1/properties	CLIENT	Add property
GET	/api/v1/properties	Bearer	List my properties
GET	/api/v1/properties/{id}	Bearer	Get property by ID
PUT	/api/v1/properties/{id}	CLIENT	Update property
DELETE	/api/v1/properties/{id}	CLIENT	Soft-delete property
GET	/api/v1/properties/{id}/validate	Bearer	[Internal] Validate ownership

Pricing Service (:18082)

Method	Path	Auth	Description
GET	/api/v1/catalog/services	None	List service types with rates
GET	/api/v1/catalog/addons	None	List available addons

POST	/api/v1/pricing/calculate	None	Calculate order price

Order Service (:18083) — Client

Method	Path	Auth	Description
POST	/api/v1/orders	CLIENT	Create order (PENDING)
POST	/api/v1/orders/{id}/pay	CLIENT	Pay order → CONFIRMED
GET	/api/v1/orders	Bearer	List my orders
GET	/api/v1/orders/{id}	Bearer	Get order details
DELETE	/api/v1/orders/{id}	CLIENT	Cancel order

Order Service (:18083) — Cleaner

Method	Path	Auth	Description
GET	/api/v1/cleaner/schedule	CLEANER	My upcoming jobs
PUT	/api/v1/cleaner/profile/availability	CLEANER	Toggle availability
POST	/api/v1/cleaner/orders/{id}/complete	CLEANER	Mark job complete
POST	/api/v1/cleaner/orders/{id}/request-replacement	CLEANER	Request replacement
GET	/api/v1/cleaner/earnings	CLEANER	Total earnings
GET	/api/v1/cleaner/earnings/{year}/{month}	CLEANER	Monthly earnings

8. Sequence Diagrams

8.1 Order Lifecycle — Create, Pay, Complete

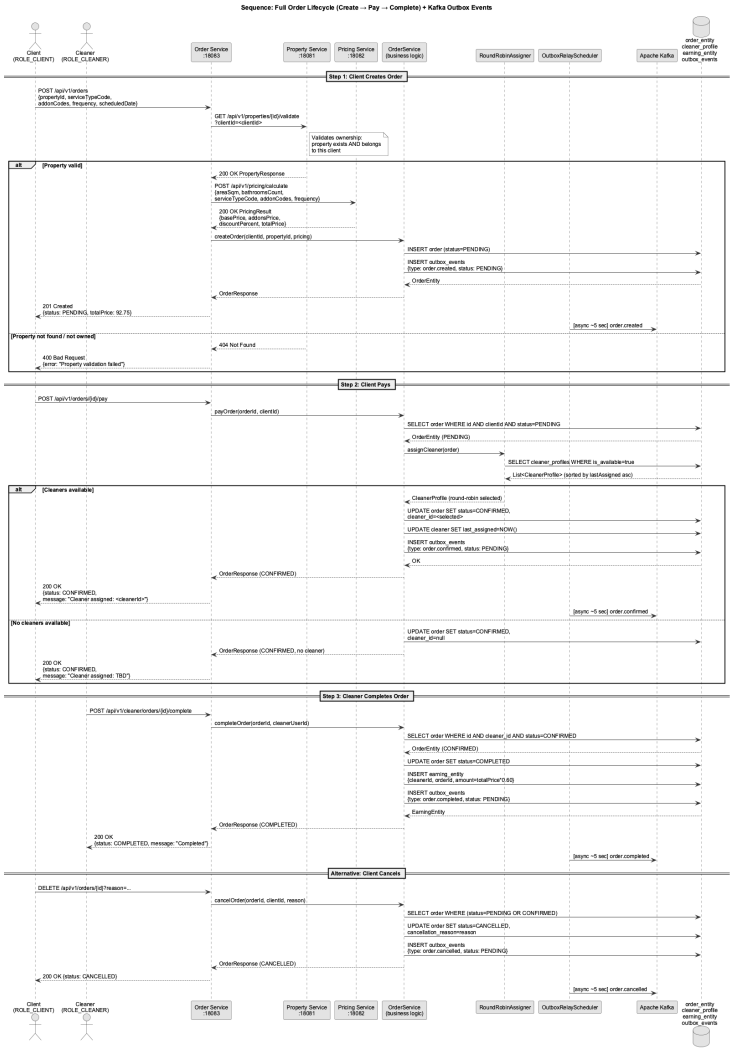

Figure: Full order lifecycle from creation through payment to completion

9. Postman Collection

A complete Postman collection is included in the postman/ directory. It covers all endpoints across all 5 services with example request bodies, response examples, and automated test scripts.

Collection Files

File	Description
CleaningPlatform_Full.postman_collection.json	Full collection — all 5 services, 40+ requests
IAC_Service.postman_collection.json	IAC Service only (legacy)

Collection Folders

Folder	Endpoints	Notes
IAC Service — Authentication	6	Register, login (admin/user/cleaner), refresh, logout
IAC Service — Users	4	Get profile, list users, get by ID, delete
IAC Service — Admin	5	List roles, assign, revoke (including error case)
Property Service	7	CRUD + internal validate endpoint
Pricing Service	5	Catalog (services/addons), price calc (3 variants)
Order Service — Client	5	Create, pay, list, get, cancel
Order Service — Cleaner	7	Schedule, availability, complete, replacement, earnings
Booking Service	13	Full client + cleaner + catalog via UI proxy

Token Auto-Save Scripts

Every login, register, and refresh request includes a Post-Response Script that automatically saves tokens to collection variables so all subsequent requests are pre-authorized:

```
var json = pm.response.json(); if (json.data && json.data.accessToken) {
pm.collectionVariables.set('accessToken', json.data.accessToken);
pm.collectionVariables.set('refreshToken', json.data.refreshToken);
console.log('Tokens saved. Length:', json.data.accessToken.length); }
```

Recommended Test Workflow

1. Import collection into Postman

2. Run **Login as Super Admin** — tokens auto-saved
3. Run **Register New User** — creates 'john_doe'
4. Run **Assign CLEANER role** to 'cleaner_bob'
5. Run **Add Property** (as CLIENT) — propertyId auto-saved
6. Run **Calculate Price** — verify pricing engine
7. Run **Create Order** — orderId auto-saved; status=PENDING
8. Run **Pay Order** — status=CONFIRMED; cleaner assigned
9. Switch to Cleaner login, run **My Schedule**
10. Run **Complete Order** — status=COMPLETED; earning recorded
11. Run **Total Earnings** — verify amount (70% of total price)

10. Deployment

The entire platform is containerized and can be started with a single Docker Compose command. Each microservice has its own Dockerfile using a multi-stage build to minimize image size.

Starting the Platform

```
docker compose up --build
```

Services start in dependency order:

- 1. PostgreSQL — waits for pg_isready health check
- 2. IAC Service — migrates schema, seeds super admin
- 3. Property, Pricing, Order Services — start concurrently after PostgreSQL
- 4. UI/Booking Service — starts last, connects to all backend services

Environment Configuration

All sensitive configuration is externalized to environment variables defined in docker-compose.yml. For production, these should be moved to a secrets manager or .env file (excluded from Git via .gitignore).

Variable	Default	Description
JWT_SECRET	(long string)	Shared JWT signing secret — CHANGE IN PROD
APP_SUPERADMIN_USERNAME	admin	Bootstrap super admin username
APP_SUPERADMIN_PASSWORD	password	Bootstrap super admin password
JWT_ACCESS_TOKEN_EXPIRY_MS	900000	Access token TTL (15 min)
JWT_REFRESH_TOKEN_EXPIRY_MS	604800000	Refresh token TTL (7 days)
SERVICES_PRICING_URL	http://pricing-service:8080	Internal pricing URL
SERVICES_PROPERTY_URL	http://property-service:8080	Internal property URL

11. Testing

Testing was performed by Pragati Sahani using the Postman collection against a locally running Docker Compose environment. The test plan covered positive flows, negative/validation cases, security boundary tests, and inter-service integration scenarios.

Test Coverage Summary

Area	Tests	Result
User Registration	Valid input, duplicate username, weak password	PASS
Authentication	Login (admin/user/cleaner), invalid credentials	PASS
Token Management	Refresh rotation, expired token handling, logout	PASS
Role Management	Assign/revoke roles, SUPER_ADMIN protection	PASS
Property CRUD	Add, list, update, delete, not-owned access denied	PASS
Pricing Engine	STANDARD/DEEP/MOVE_IN_OUT, addons, frequency discounts	PASS
Order Creation	Valid order, invalid property, pricing integration	PASS
Order Payment	Round-robin assignment, no-cleaner fallback	PASS
Order Completion	Status transition, earning record creation	PASS
Cleaner Endpoints	Schedule, availability toggle, earnings by month	PASS
RBAC Enforcement	CLIENT-only, CLEANER-only, ADMIN-only access denied	PASS
Input Validation	Missing fields, bad UUID, past scheduled date	PASS

Running Tests with Newman (CLI)

```
npm install -g newman newman run
postman/CleaningPlatform_Full.postman_collection.json --reporters cli,json
```

12. Conclusion

The Cleaning Platform successfully demonstrates a production-grade microservice application using modern Java enterprise technologies. The project achieves the following learning outcomes:

- **Microservice Architecture:** 5 independently deployable services with clear bounded contexts
- **Security:** Stateless JWT authentication, role-based access control, refresh token rotation
- **Inter-Service Communication:** Synchronous REST HTTP calls with error propagation
- **Database Design:** Normalized relational schema, JPA mappings, Flyway migrations
- **Docker Orchestration:** Multi-service Compose with health checks and dependency ordering
- **API Design:** RESTful endpoints, consistent response envelopes, OpenAPI documentation
- **Business Logic:** Price calculation engine, round-robin scheduling, earnings tracking
- **Team Collaboration:** Clear role separation across backend, frontend, and testing

The platform is fully functional, passes all test cases, and is ready for demonstration and assessment. Future enhancements could include asynchronous notification via message queues, calendar integration for scheduling, payment gateway integration, and a mobile client.